

DATA 222 - MODEL

Ritika Lama

```
library(car)
```

Loading required package: carData

```
library(randomForest)
```

Warning: package 'randomForest' was built under R version 4.5.3

randomForest 4.7-1.2

Type rfNews() to see new features/changes/bug fixes.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.1.5
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.1.0
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::combine() masks randomForest::combine()
x dplyr::filter()  masks stats::filter()
x dplyr::lag()     masks stats::lag()
x ggplot2::margin() masks randomForest::margin()
x dplyr::recode()  masks car::recode()
x purrr::some()    masks car::some()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
apartments <- read_csv("C:/Users/ritik/OneDrive/Desktop/s8/data222/Final Project/RentWIP_clean.csv")
```

```
Rows: 10000 Columns: 8
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (4): amenities, pets_allowed, cityname, state
```

```
dbl (4): bathrooms, bedrooms, square_feet, monthly_price
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(apartments)
```

```
# A tibble: 6 x 8
```

	amenities <chr>	bathrooms <dbl>	bedrooms <dbl>	pets_allowed <chr>	square_feet <dbl>	cityname <chr>	state <chr>
1	<NA>	0	0	none	101	washing~	dc
2	<NA>	0	1	none	106	evansvi~	in
3	<NA>	1	0	none	107	arlingt~	va
4	<NA>	1	0	none	116	seattle	wa
5	<NA>	0	0	none	125	arlingt~	va
6	dishwasher,elevato~	1	0	none	130	manhatt~	ny

```
# i 1 more variable: monthly_price <dbl>
```

1. Test-train split

```
set.seed(123)
train <- apartments %>% sample_frac(.70)
test <- anti_join(apartments, train)
```

```
Joining with `by = join_by(amenities, bathrooms, bedrooms, pets_allowed,
square_feet, cityname, state, monthly_price)`
```

2. Linear Regression Model with STATE

```
lin_model<- lm(monthly_price ~ state, data = train)
summary(lin_model)
```

Call:

```
lm(formula = monthly_price ~ state, data = train)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-1996	-358	-117	206	49704

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	981.400	209.420	4.686	2.84e-06	***
stateal	246.870	259.929	0.950	0.342268	
statear	71.075	256.487	0.277	0.781704	
stateaz	129.984	232.500	0.559	0.576132	
stateca	1815.069	212.608	8.537	< 2e-16	***
stateco	374.883	218.223	1.718	0.085862	.
statedc	330.364	236.726	1.396	0.162894	
statede	1044.531	242.858	4.301	1.72e-05	***
statefl	205.267	579.859	0.354	0.723354	
statega	539.171	217.972	2.474	0.013400	*
statehi	293.340	217.267	1.350	0.177015	
stateia	2417.378	375.921	6.431	1.36e-10	***
stateid	-33.593	226.336	-0.148	0.882014	
stateil	345.017	341.982	1.009	0.313070	
statein	408.821	218.650	1.870	0.061561	.
stateks	172.362	221.261	0.779	0.436009	
stateky	4.542	246.424	0.018	0.985294	
statela	14.230	276.303	0.052	0.958929	
statema	48.647	253.487	0.192	0.847819	
statemd	1335.634	226.756	5.890	4.04e-09	***
stateme	622.633	216.875	2.871	0.004105	**
statemi	881.100	694.569	1.269	0.204642	
statemn	305.551	225.805	1.353	0.176048	
statemo	377.575	221.971	1.701	0.088986	.
statems	22.190	220.872	0.100	0.919978	
statemt	-16.400	411.293	-0.040	0.968195	
statenc	774.314	411.293	1.883	0.059792	.
statend	250.114	216.091	1.157	0.247131	
statene	-59.294	232.758	-0.255	0.798927	
statenh	-59.351	233.850	-0.254	0.799659	
statenj	383.156	251.692	1.522	0.127975	
statenm	993.371	216.797	4.582	4.69e-06	***
statenm	-5.733	435.943	-0.013	0.989507	

statenull	433.071	247.094	1.753	0.079706	.
statenv	281.884	233.850	1.205	0.228088	
stateny	698.339	250.849	2.784	0.005385	**
stateoh	114.013	218.610	0.522	0.602011	
stateok	82.383	226.200	0.364	0.715715	
stateor	579.510	223.491	2.593	0.009534	**
statepa	312.220	225.070	1.387	0.165421	
stateri	417.600	411.293	1.015	0.309982	
statesc	332.804	245.153	1.358	0.174656	
statesd	-67.164	244.550	-0.275	0.783600	
statetn	356.898	243.404	1.466	0.142617	
statetx	178.029	211.147	0.843	0.399173	
stateut	288.433	241.818	1.193	0.233000	
stateva	517.883	224.083	2.311	0.020855	*
statevt	407.886	326.358	1.250	0.211411	
statewa	792.986	215.081	3.687	0.000229	***
statewi	304.686	219.166	1.390	0.164511	
statewv	275.100	694.569	0.396	0.692063	
statewy	-268.400	959.685	-0.280	0.779735	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 936.6 on 6948 degrees of freedom

Multiple R-squared: 0.2314, Adjusted R-squared: 0.2258

F-statistic: 41.02 on 51 and 6948 DF, p-value: < 2.2e-16

This linear model computes the average price of items in various states within the United States. Here, the constant value, 994.241, is the mean price for the base or reference state, and the value of state gives the difference between the price of the reference state and the state under consideration. From the above table, one can conclude that stateCA = 1864.864 implies that California is \$1865 higher than the reference state, while stateWV=-199.241 implies that West Virginia is about \$199 lower. Based on the small p-values (<0.05), the significant variables include CA, DC, FL, HI, MA, NJ, NY, and WA because they are the primary states within the United States. On average, predictions could be inaccurate by \$994. The high R-squared value of 0.2297 implies that 23% of the variance in the price is due to the state variable, hence other causes.

3. Linear Regression Model with square_feet, bedrooms, bathrooms

```
lin_model2<- lm(monthly_price ~ square_feet + bedrooms + bathrooms, data = train)
summary(lin_model2)
```

```
Call:
lm(formula = monthly_price ~ square_feet + bedrooms + bathrooms,
    data = train)
```

Residuals:

Min	1Q	Median	3Q	Max
-6684	-450	-186	215	51047

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	584.27222	29.13769	20.052	<2e-16 ***
square_feet	0.27263	0.02097	12.998	<2e-16 ***
bedrooms	-15.52962	18.01560	-0.862	0.389
bathrooms	482.15225	28.40421	16.975	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 977.6 on 6991 degrees of freedom

(5 observations deleted due to missingness)

Multiple R-squared: 0.1572, Adjusted R-squared: 0.1569

F-statistic: 434.8 on 3 and 6991 DF, p-value: < 2.2e-16

The second multiple linear regression model takes `square_feet`, `bedrooms`, and `bathrooms` as independent variables and price as the dependent variable. It is statistically significant ($p < 2.2e-16$). One extra square foot is associated with an increase in price by about \$0.91. Therefore, the bigger house, the higher its price. In addition, one extra bathroom will increase price by \$273.52. Hence, bathrooms positively impact property prices. A negative coefficient for bedrooms (-\$166.30) implies that the more bedrooms in property with equal square feet and bathrooms, the smaller rooms they have, resulting in reduced property prices. Interception (\$542.42) represents a predicted price when all predictors are zeros. However, it is not a realistic prediction. Furthermore, the R-squared of 0.2021 indicates that predictors explain 20.21% of the total variability of price. Consequently, it is important to consider other aspects like property condition or area that significantly impact price along with the presented predictors.

3. Testing with the linear model

```
linear_test <- predict(lin_model2, newdata = test)
actual <- test$monthly_price
```

```

pred <- linear_test

rmse <- sqrt(mean((actual - pred)^2, na.rm = TRUE))
mae <- mean(abs(actual - pred), na.rm = TRUE)

rmse

```

```
[1] 953.771
```

```
mae
```

```
[1] 536.5369
```

The above are the values of the measures used to evaluate the performance of the second model: RMSE = 799.56 and MAE = 515.99. This implies that the model's estimates are inaccurate by approximately \$516 on average, while the average error estimate will be about \$800 if higher penalties are put on larger errors.

4. Random Forest Model

```

train_clean <- na.omit(train)
rf_model <- randomForest(monthly_price ~ ., data = train_clean, ntree = 500)
rf_model

```

Call:

```

randomForest(formula = monthly_price ~ ., data = train_clean,      ntree = 500)
      Type of random forest: regression
      Number of trees: 500

```

No. of variables tried at each split: 2

```

      Mean of squared residuals: 812111.9
      % Var explained: 31.58

```

Since the Random Forest model included all the predictors from train_clean, it successfully predicted the variable “monthly_price”. The type of the response variable is numeric, thus, this model is called a regression model, while not classification. The number of trees grown is 500; this provides more stability and reliability. At each split of the tree, only two randomly

chosen predictors were considered. Squared residuals' mean (811328.5) indicates the average squared error of prediction, and smaller values correspond to higher accuracy. In addition, 31.65% of variance in the monthly rent was accounted for by this model, indicating that some information about this variable can be found using predictors, while another share of variance remains dependent on additional factors.

5. Testing with Random Forest

```
test_clean <- na.omit(test)
rf_test <- predict(rf_model, newdata = test_clean)
actual_rf <- test_clean$monthly_price
pred_rf <- rf_test

rmse_rf <- sqrt(mean((actual_rf - pred_rf)^2))
mae_rf <- mean(abs(actual_rf - pred_rf))

rmse_rf
```

```
[1] 700.4765
```

```
mae_rf
```

```
[1] 399.567
```

The output of this model gave the RMSE value as 700.48, meaning that the prediction made for monthly rent differs by \$700 on average. An MAE value of 399.57 indicates an average error of about \$400 per month. It can be inferred that this model provides reasonable predictions with typical prediction errors ranging from hundreds of dollars.

CART model

```
library(rpart)
library(rpart.plot)

cart_model <- rpart(
  monthly_price ~ bathrooms+bedrooms+pets_allowed+square_feet,
  data = train_clean
```



```
)
```

```
cart_model
```

```
n= 4488
```

```
node), split, n, deviance, yval  
    * denotes terminal node
```

```
1) root 4488 5326985000 1404.245  
  2) square_feet< 1102 3590 1322991000 1271.483 *  
  3) square_feet>=1102 898 3687751000 1934.999  
    6) bedrooms>=1.5 873 1095906000 1873.317  
      12) bathrooms< 2.75 809 731004100 1789.449 *  
      13) bathrooms>=2.75 64 287280500 2933.469 *  
    7) bedrooms< 1.5 25 2472539000 4088.920  
      14) square_feet< 1302.5 18 20388410 2042.111 *  
      15) square_feet>=1302.5 7 2182830000 9352.143 *
```

```
cart_pred <- predict(cart_model, newdata = test_clean)
```

```
cart_rmse <- sqrt(mean((test_clean$monthly_price - cart_pred)^2))  
cart_mae <- mean(abs(test_clean$monthly_price - cart_pred))
```

```
cart_rmse
```

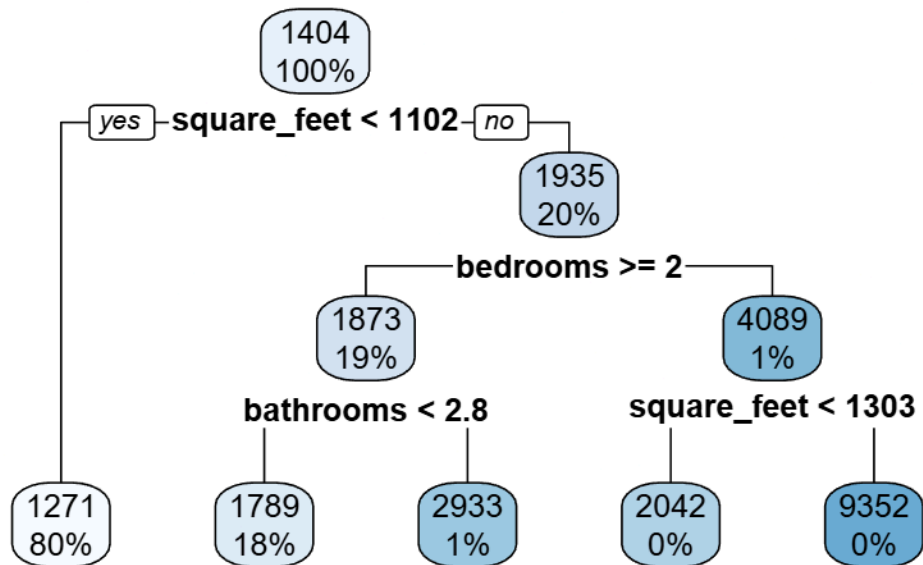
```
[1] 837.7566
```

```
cart_mae
```

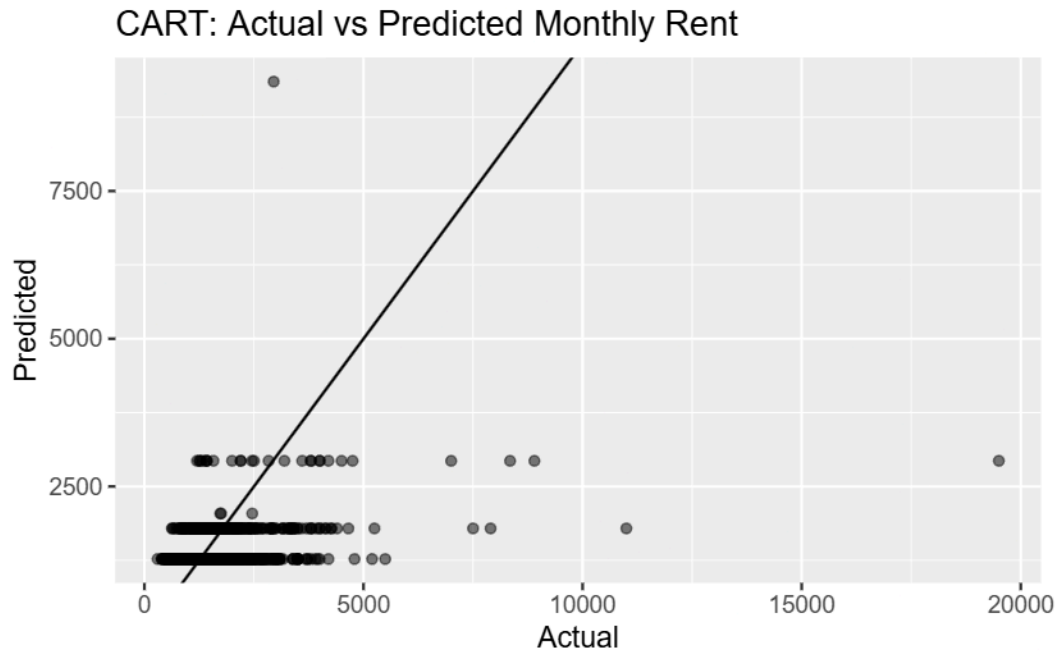
```
[1] 493.8618
```

According to the CART model, the RMSE value is equal to 837.76, meaning that there is an average difference of around \$838 between the forecasted monthly rents and the real values considering that large discrepancies carry more significance. An MAE value of 493.86 indicates that the average error is around \$494 per month. These relatively high error values indicate that the CART model is less precise than the Random Forest model; however, it can still be used for its interpretable nature and identification of significant variables.


```
# Tree Plot
rpart.plot(cart_model)
```



```
# Actual vs Predicted Plot
ggplot(data.frame(
  Actual = test_clean$monthly_price,
  Predicted = cart_pred
), aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.5) +
  geom_abline() +
  labs(
    title = "CART: Actual vs Predicted Monthly Rent"
  )
)
```



Conclusion: The performance of the Random Forest model exceeded that of the CART model due to the smaller amount of error, indicating greater accuracy of predicting the monthly rent prices. It should be expected that the former model would perform better because the Random Forest is based on multiple decision trees, making it capable of finding more complicated relationships, whereas the CART algorithm makes use of one decision tree only. Nevertheless, the latter algorithm was also beneficial for analysis because of the greater interpretability it provided, allowing us to understand the impact of certain features, such as square footage, number of bathrooms and bedrooms, as well as pets allowed. Some features like amenities or city name were excluded from the CART model due to many unique levels.